$$x_{k+1} = x_k - \gamma \nabla f(x_k)$$

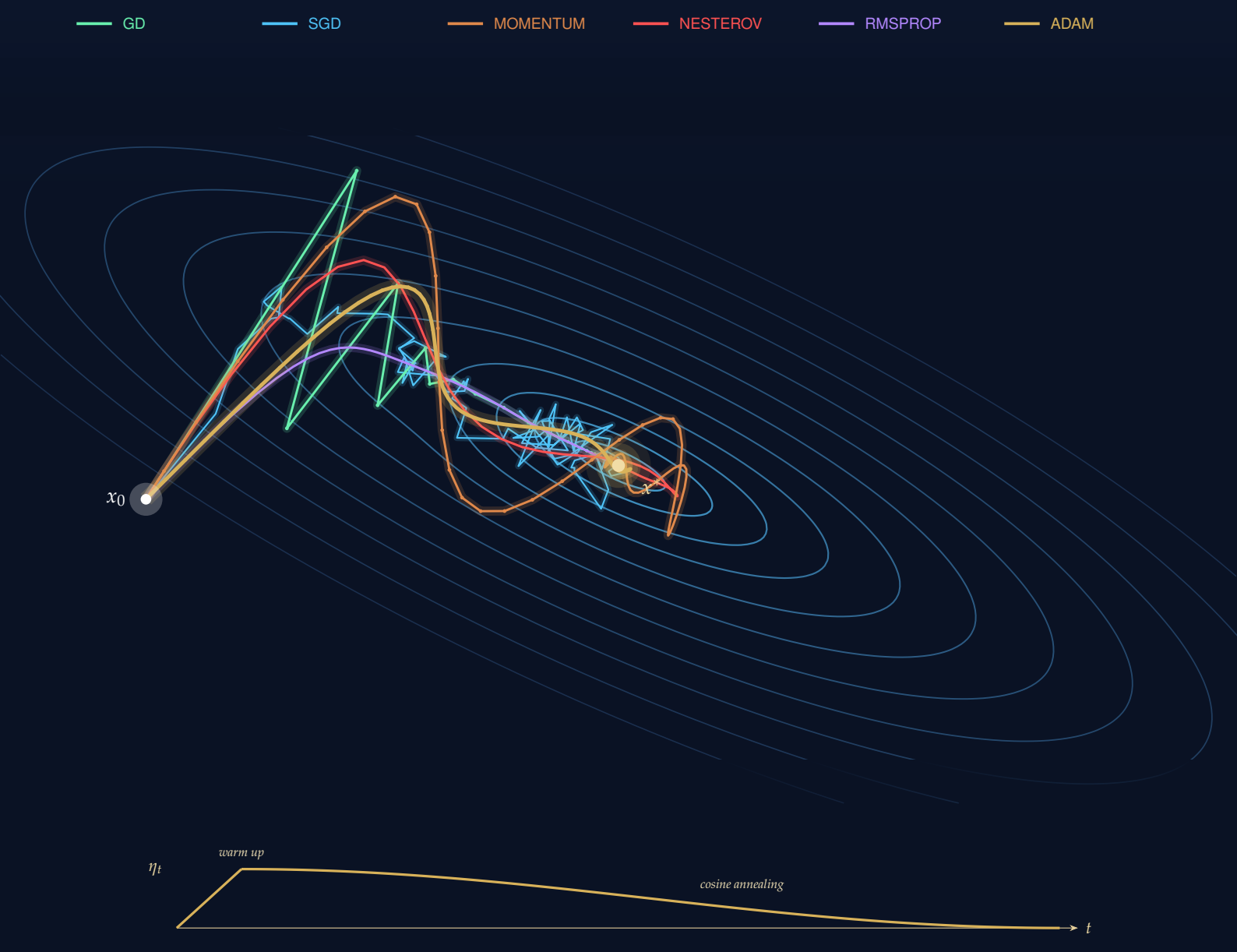
$$v_t = \beta v_{t-1} + (1-\beta)\nabla f_t$$

A HANDWRITTEN THEORY, FROM THE GRADIENT TO THE MINIMUM

# THE ENCHIRIDION

## *for AI Optimizers Theory*

CONVEXITY · L-SMOOTHNESS · GRADIENT DESCENT · SGD · MOMENTUM · NESTEROV
RMSPROP · ADAM · ADAMW · ADAMAX · AMSGRAD · QHADAM · YOGI · SCHEDULES




REDA LOURIKI

# *Preface*

EVERY model we train, from a small linear regression to a large language model, learns in the same way: an optimizer follows the gradient of a loss, one step at a time, hoping to reach its minimum. This book is about those steps.

It started, like the first *Enchiridion*, as handwritten notes. From May to June 2026, I wrote by hand the theory of the optimizers used in machine learning and deep learning: first the mathematical tools (convexity and L-smoothness), then Gradient Descent and its proof of convergence, and then every idea that was added to it to make it faster, more stable, or more robust: Stochastic Gradient Descent, Momentum, Nesterov, RMSProp, Adam, AdamW, and the other versions of Adam, until the learning rate schedules that decide how fast we move.

An *enchiridion* is, in Greek, "something that is held in the hand": a handbook. This is the second one of the series, after *The Enchiridion for Tree Based Algorithms Theory*.

## *Who made this book*

**Author.** The content of this book (the explanations, the proofs, the examples and the way of telling them) was created by **Reda Louriki**, in handwritten notes.

**Typesetting.** The handwritten notes were transcribed and turned into LaTeX by an AI, **Claude Opus 5.5** (Anthropic). The author's wording and examples were kept; only grammatical errors and mathematical errors were corrected. The part of the last chapter that the notes left unfinished (the warm up) was completed in the same spirit.

Every time a correction changes the meaning of what was written in the notes (a formula, an index, a number, a name, or a conclusion), it is signalled with a footnote marked **Editor's note**, so you can always see what the notes said and why it was changed. The list of all these changes is gathered at the end of the book, in *Notes on this Edition*.

## *How to read this book*

The notes were written with several colours of ink, and each colour had a job. The book keeps the same code:

| | |
|---|---|
| *Red* | the titles of the chapters and the warnings ⚠ |
| *Green* | definitions, theorems, "Problem" and "Solution" |
| *"Gold"* | the motto of each optimizer, in one sentence |
| *Blue* | demonstrations and remarks |
| *Purple* | the questions you should be asking yourself at that moment |
| *Gold boxes* | the sentences worth remembering |

The chapters follow the logic of the ideas: each optimizer is born from a problem of the previous one. Every chapter starts with that problem, and ends with its solution. So the best way to read this book is in order.

The prerequisites are some linear algebra (inner products, norms, eigenvalues), derivatives and gradients, some probability (expectation, variance), and the curiosity to ask "why?" at every line. The cover shows the idea of the whole book: the real trajectories of six of these optimizers, computed on the same loss landscape, from the same starting point, racing to the same minimum.

*Reda Louriki*
*2026*

# Contents

**Preface** **i**

**I Prerequisite** **1**
I.1 Convexity . . . 1
I.2 L-Smooth Functions . . . 3
I.3 Convex and L-Smooth Functions . . . 5

**II Gradient Descent** **7**

**III Stochastic Gradient Descent** **11**
III.1 Key Improvements . . . 13

**IV Momentum and Nesterov** **15**
IV.1 Exponentially Weighted Average . . . 15
IV.2 Momentum . . . 17
IV.3 Nesterov . . . 19

**V RMSProp** **21**

**VI Adam and AdamW** **25**
VI.1 Adam . . . 25
VI.2 AdamW . . . 27

**VII Other Adam Versions and Improvements** **29**
VII.1 AdaMax . . . 29
VII.2 AMSGrad . . . 31
VII.3 QHAdam . . . 31
VII.4 Yogi . . . 32

**VIII Learning Rate Schedule** **35**

**Notes on this Edition** **41**

CHAPTER

# I

# Prerequisite

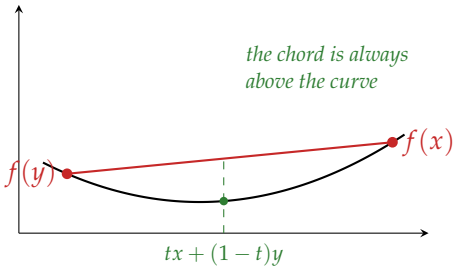
BEFORE talking about optimizers, we need two tools. Every proof of this book will use them: *convexity* and *L-smoothness*.[1]

## I.1 Convexity

*Def 1:*

Let $f : \mathbb{R}^d \to \mathbb{R}$. $f$ is convex iff $\forall x, y \in \mathbb{R}^d$, $\forall t \in ]0, 1[$:

$$f\big(tx + (1-t)y\big) \leq tf(x) + (1-t)f(y)$$




*Th 1:*

Let $f : \mathbb{R}^d \to \mathbb{R}$ be differentiable.[2] $\forall x, y \in \mathbb{R}^d$: $f$ is convex iff:

$$\big\langle \nabla f(y),\, x - y \big\rangle \leq f(x) - f(y)$$

[1] **Editor's note.** This short introduction was added to open the book; the notes start directly with "I. Convexity".
[2] **Editor's note.** The notes don't state it, but the theorem needs $f$ to be differentiable, since it uses $\nabla f$.

*Demo:*

$$\begin{aligned} f \text{ is cvx} &\iff f\big(tx+(1-t)y\big) \le tf(x)+(1-t)f(y) \\ &\iff f\big(y+t(x-y)\big) \le f(y)+t\big(f(x)-f(y)\big) \\ &\iff \frac{f\big(y+t(x-y)\big)-f(y)}{t} \le f(x)-f(y) \qquad (1) \end{aligned}$$

We know that, by using Taylor's development:

$$f(l+h) = f(l) + f'(l)\,h + o(h)$$

But if $l \in \mathbb{R}^d$ with $d \in \mathbb{N}^* \setminus \{1\}$, then:

$$f(l+h) = f(l) + \underbrace{\langle \nabla f(l), h \rangle}_{\text{since } \nabla f(l)\cdot h \text{ has no meaning}} + o\big(\|h\|\big) \qquad [3]$$

Let $l = y$ and $h = t(x-y)$, then:

$$\begin{aligned} &\iff f\big(y+t(x-y)\big) = f(y) + \big\langle \nabla f(y), t(x-y) \big\rangle + o(t) \\ &\iff \frac{f\big(y+t(x-y)\big)-f(y)}{t} = \big\langle \nabla f(y), x-y \big\rangle + \frac{o(t)}{t} \\ &\iff \lim_{t\to 0} \frac{f\big(y+t(x-y)\big)-f(y)}{t} = \big\langle \nabla f(y), x-y \big\rangle \qquad (2) \end{aligned}$$

$\longrightarrow$ This is the directional derivative; we will take this statement without proving it.

$(2) \to (1)$:

$$\begin{aligned} \lim_{t\to 0} \frac{f\big(y+t(x-y)\big)-f(y)}{t} &\le f(x)-f(y) \\ \iff \big\langle \nabla f(y), x-y \big\rangle &\le f(x)-f(y) \\ \iff \big\langle \nabla f(y), x-y \big\rangle + f(y) &\le f(x) \qquad \blacksquare [4] \end{aligned}$$

*I call this formula "the global minimum formula".*

Because, let $y = x^*$, with $x^*$ a minimum (the point we are interested in).[5] Then $\nabla f(x^*) = 0$, and so $\big\langle \nabla f(x^*), x - x^* \big\rangle = 0$, which leads to:

$$f(x^*) \le f(x) \qquad \forall x \in \mathbb{R}^d$$

[3] **Editor's note.** The notes write the remainder as $o(h^2)$. A first-order Taylor expansion has a remainder $o(h)$ (or $O(h^2)$ if $f$ is twice differentiable); $o(h^2)$ would be a second-order statement. The conclusion does not change: after dividing by $t$, the remainder still goes to 0.

[4] **Editor's note.** This proves the direct implication (convex $\Rightarrow$ inequality). The converse is also true: apply the inequality at the point $z = tx + (1-t)y$, once towards $x$ and once towards $y$, multiply by $t$ and $1-t$, and sum.

[5] **Editor's note.** The notes say "a min that we want to prove unicity". What the formula proves is that a point where the gradient vanishes is a *global* minimum; it does not prove that this minimum is unique (for that, $f$ must be *strictly* convex).

It's the **global minimum**.

*Th 2:*

Let $f : \mathbb{R}^d \to \mathbb{R}$ be convex and twice differentiable; then $\forall x \in \mathbb{R}^d$, for $\lambda$ an eigenvalue of $\nabla^2 f(x)$: $\lambda \geq 0$.

*Demo:*

From Th 1:

$$\langle \nabla f(y), x - y \rangle \leq f(x) - f(y) \qquad (1)$$

By applying it on $x$:

$$\langle \nabla f(x), y - x \rangle \leq f(y) - f(x) \qquad (2)$$

$(1) + (2)$:

$$\begin{aligned} \langle \nabla f(x), y - x \rangle + \langle \nabla f(y), x - y \rangle &\leq 0 \\ 0 &\leq \langle \nabla f(x), x - y \rangle - \langle \nabla f(y), x - y \rangle \\ 0 &\leq \langle \nabla f(x) - \nabla f(y), x - y \rangle \qquad (3) \end{aligned}$$

Let's develop this expression:[6]

$$\begin{aligned} \langle \nabla^2 f(x) u, u \rangle &= \lim_{t \to 0} \left\langle \frac{\nabla f(x + tu) - \nabla f(x)}{t}, u \right\rangle \\ &= \lim_{t \to 0} \left\langle \frac{\nabla f(x + tu) - \nabla f(x)}{t}, \frac{(x + tu) - x}{t} \right\rangle \\ &= \lim_{t \to 0} \frac{1}{t^2} \underbrace{\langle \nabla f(x + tu) - \nabla f(x), (x + tu) - x \rangle}_{\geq 0 \text{ from (3)}} \geq 0 \end{aligned}$$

So:

$$\boxed{\langle \nabla^2 f(x) u, u \rangle \geq 0}$$

Let $u$ be an eigenvector of $\nabla^2 f(x)$, so $\nabla^2 f(x) u = \lambda u$. Then:

$$\langle \lambda u, u \rangle \geq 0 \iff \lambda \langle u, u \rangle \geq 0 \iff \lambda \|u\|^2 \geq 0 \implies \boxed{\lambda \geq 0}$$

So for any eigenvalue $\lambda$ of $\nabla^2 f(x)$, we are sure that it's non-negative. ■

## I.2 *L-Smooth Functions*

*Def 2: L-Lipschitz function:*

Let $f : \mathbb{R}^d \to \mathbb{R}^m$. $f$ is L-Lipschitz iff $\exists L > 0, \forall x, y \in \mathbb{R}^d$:[7]

[6] **Editor's note.** The notes omit the limit $t \to 0$ in the definition of the Hessian; it was added.

[7] **Editor's note.** The notes write $\exists L \in ]0, 1[$ (here and in Th 3). The Lipschitz constant can be any positive number; $L < 1$ is the special case of a *contraction*. In the chapter on Gradient Descent the learning rate is $1/L$, which would be larger than 1 if $L < 1$ was required, so the general case is needed.

$$\|f(x) - f(y)\| \leq L\|x - y\|$$

*Def 3: L-Smooth:*

Let $f : \mathbb{R}^d \to \mathbb{R}$. $f$ is L-smooth iff $f$ is differentiable and $\nabla f : \mathbb{R}^d \to \mathbb{R}^d$ is L-Lipschitz; $\forall x, y \in \mathbb{R}^d$:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$$

*Th 3:*

If $f$ is L-smooth, then $\forall x, y \in \mathbb{R}^d$:[8]

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2$$

*Demo:*

Let $\phi(t) = f\big(x + t(y - x)\big)$. Then, by applying the fundamental theorem of calculus:

$$\phi(1) - \phi(0) = \int_0^1 \phi'(t)\, dt$$

$$\begin{aligned}
&\iff f(y) - f(x) = \int_0^1 \big\langle \nabla f\big(x + t(y - x)\big),\, y - x \big\rangle dt \\
&\iff f(y) - f(x) = \langle \nabla f(x), y - x \rangle + \int_0^1 \big\langle \nabla f\big(x + t(y - x)\big) - \nabla f(x),\, y - x \big\rangle dt \\
&\implies f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \int_0^1 \big\|\nabla f\big(x + t(y - x)\big) - \nabla f(x)\big\|\, \|y - x\|\, dt \\
&\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \text{(Cauchy–Schwarz)} \\
&\implies f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \int_0^1 L\, t\, \|y - x\|^2\, dt \qquad (\nabla f \text{ is L-Lipschitz}) \\
&\implies f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2 \qquad \blacksquare^{9}
\end{aligned}$$

[8] **Editor's note.** The notes write "$f$ is L-smooth *iff*". The inequality (called the *descent lemma*) is implied by L-smoothness, but the converse is false in general (a concave function satisfies it without being L-smooth). The converse holds if $f$ is also convex.

[9] **Editor's note.** The notes write " $\iff$ " at every line; from the Cauchy–Schwarz step on, the lines are only implications ( $\implies$ ).

## I.3 *Convex and L-Smooth Functions*

*Th 4:*

If a function is convex and L-smooth, then:[10]

$$\begin{cases} \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \leq f(y) - f(x) - \langle \nabla f(x), y - x \rangle & (1) \\ \frac{1}{L}\|\nabla f(y) - \nabla f(x)\|^2 \leq \langle \nabla f(y) - \nabla f(x), y - x \rangle & (2) \end{cases}$$

*Demo:*

$$f(x) - f(y) = f(x) - f(z) + f(z) - f(y)$$

We know that (convexity, Th 1):

$$\begin{aligned} \langle \nabla f(x), z - x \rangle &\leq f(z) - f(x) \\ \langle \nabla f(x), x - z \rangle &\geq f(x) - f(z) \qquad (*) \end{aligned}$$

And (L-smoothness, Th 3):

$$\begin{aligned} f(z) &\leq f(y) + \langle \nabla f(y), z - y \rangle + \frac{L}{2}\|z - y\|^2 \\ \implies f(z) - f(y) &\leq \langle \nabla f(y), z - y \rangle + \frac{L}{2}\|z - y\|^2 \qquad (**) \end{aligned}$$

By summing $(*)$ and $(**)$, we have:

$$\begin{aligned} f(x) - f(y) &\leq \langle \nabla f(x), x - z \rangle + \langle \nabla f(y), z - y \rangle + \frac{L}{2}\|z - y\|^2 \\ f(x) - f(y) &\leq \underbrace{\langle \nabla f(x), x \rangle - \langle \nabla f(x), z \rangle + \langle \nabla f(y), z \rangle - \langle \nabla f(y), y \rangle + \frac{L}{2}\|z - y\|^2}_{g(z)} \end{aligned}$$

This inequality is valid for every $z \in \mathbb{R}^d$. Let's take the perfect one (the min):

$$\nabla_z g(z) = -\nabla f(x) + \nabla f(y) + L\,(z - y) = 0 \iff \boxed{z - y = -\frac{1}{L}\big(\nabla f(y) - \nabla f(x)\big)} \quad (***)^{11}$$

[10] **Editor's note.** The notes call it "Def 4". It is a property that we prove (so a theorem), and it was renamed accordingly. In the notes, the right-hand side of (2) is written $\langle \nabla f(x) - \nabla f(y), y - x \rangle$ on this line and correctly $\langle \nabla f(y) - \nabla f(x), y - x \rangle$ at the end of the demo; the sign was fixed here.

$$
\begin{aligned}
&\iff f(x) - f(y) \leq \langle \nabla f(x), x - z \rangle + \langle \nabla f(y), z - y \rangle + \frac{L}{2}\|z - y\|^2 \\
&\iff f(x) - f(y) \leq \langle \nabla f(x), x - y \rangle + \langle \nabla f(y) - \nabla f(x), z - y \rangle + \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \\
&\iff f(x) - f(y) \leq \langle \nabla f(x), x - y \rangle + \left\langle \nabla f(y) - \nabla f(x), -\tfrac{1}{L}(\nabla f(y) - \nabla f(x)) \right\rangle \\
&\qquad\qquad + \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \\
&\iff f(x) - f(y) \leq \langle \nabla f(x), x - y \rangle - \frac{1}{L}\|\nabla f(y) - \nabla f(x)\|^2 + \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \\
&\iff f(x) - f(y) \leq \langle \nabla f(x), x - y \rangle - \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \\
&\iff \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \leq \langle \nabla f(x), x - y \rangle + f(y) - f(x) \\
&\iff \frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \leq f(y) - f(x) - \langle \nabla f(x), y - x \rangle \qquad \blacksquare
\end{aligned}
$$

*Demo for (2):*

We write (1), then (1) again with $x$ and $y$ swapped:

$$
\begin{cases}
\frac{1}{2L}\|\nabla f(y) - \nabla f(x)\|^2 \leq f(y) - f(x) - \langle \nabla f(x), y - x \rangle \\
\frac{1}{2L}\|\nabla f(x) - \nabla f(y)\|^2 \leq f(x) - f(y) - \langle \nabla f(y), x - y \rangle
\end{cases}
$$

By summing:

*Co-coercivity*

$$
\frac{1}{L}\|\nabla f(y) - \nabla f(x)\|^2 \leq \langle \nabla f(y) - \nabla f(x), y - x \rangle
$$

[11] **Editor's note.** The notes write the optimality condition with norms: $-\nabla f(x) + \nabla f(y) + L\|z - y\| = 0$ and $\|z - y\| = \frac{1}{L}\|\nabla f(y) - \nabla f(x)\|$. The gradient of $g$ is a vector, so the condition is a vector equation; its direction (the minus sign) is needed in the next step.

CHAPTER

# II

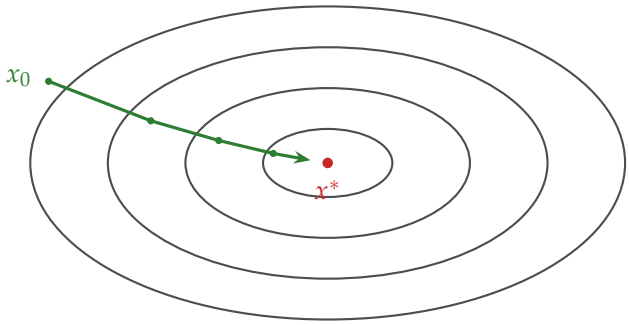
# Gradient Descent

*Def: GD: "Follow the steepest slope directly."*

The gradient descent is an algorithm defined by the sequence $(x_k)$:

$$x_{k+1} = x_k - \gamma \nabla f(x_k)$$

and $\gamma$ is called the **learning step** (the learning rate).




*Th: Proof of convergence:*

The gradient descent is defined as:

$$x_{k+1} = x_k - \gamma \nabla f(x_k)$$

Let's determine the optimal range for $\gamma$. $f$ is convex and L-smooth, and so (Th 3):

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2}\|y - x\|^2$$

Let $y = x - \gamma \nabla f(x) \implies y - x = -\gamma \nabla f(x)$. So:[1]

$$f(y) \leq f(x) - \gamma \|\nabla f(x)\|^2 + \frac{L}{2}\gamma^2 \|\nabla f(x)\|^2$$
$$f(y) \leq f(x) + \gamma \left(\frac{L}{2}\gamma - 1\right) \|\nabla f(x)\|^2$$

$\gamma \left(\frac{L}{2}\gamma - 1\right) < 0$, so that the sequence $f(x_k)$ decreases. $\gamma > 0$, and so:

$$\frac{L}{2}\gamma - 1 < 0 \implies \gamma < \frac{2}{L} \implies 0 < \gamma < \frac{2}{L}$$

[1] **Editor's note.** The notes write the last term $\frac{L}{2}\gamma^2 \|f(x)\|^2$; it is $\|\nabla f(x)\|^2$.

Let $\gamma = \frac{1}{L}$ (we can also choose $\gamma = \frac{1.5}{L}$).

So the new expression of GD:

$$\begin{aligned} & x_{k+1} = x_k - \frac{1}{L}\nabla f(x_k) \\ \iff & x_{k+1} - x^* = x_k - x^* - \frac{1}{L}\nabla f(x_k) \\ & \|x_{k+1} - x^*\|^2 = \|x_k - x^*\|^2 - \frac{2}{L}\langle \nabla f(x_k), x_k - x^* \rangle + \frac{1}{L^2}\|\nabla f(x_k)\|^2 \end{aligned}$$

From the definition of L-smooth functions, the co-coercivity (Th 4), with $\nabla f(x^*) = 0$:

$$\begin{aligned} & \frac{1}{L}\|\nabla f(x_k) - \nabla f(x^*)\|^2 \leq \langle \nabla f(x_k) - \nabla f(x^*), x_k - x^* \rangle \\ \iff & \frac{1}{L}\|\nabla f(x_k)\|^2 \leq \langle \nabla f(x_k), x_k - x^* \rangle \\ \iff & -\frac{2}{L^2}\|\nabla f(x_k)\|^2 \geq -\frac{2}{L}\langle \nabla f(x_k), x_k - x^* \rangle \end{aligned}$$

By replacing this expression:[2]

$$\begin{aligned} & \|x_{k+1} - x^*\|^2 \leq \|x_k - x^*\|^2 - \frac{1}{L^2}\|\nabla f(x_k)\|^2 \\ \text{So} \quad & \|x_{k+1} - x^*\| \leq \|x_k - x^*\| \\ \iff & \boxed{\|x_{k+1} - x^*\| \leq \|x_0 - x^*\|} \end{aligned}$$

So the distance between the GD sequence $(x_k)$ and $x^*$ is decreasing: at each step, GD gets closer to (or at least never moves away from) the minimum.[3]

*Th: Proof for the speed of convergence (convex L-smooth functions):*

$$\begin{aligned} & f(x_{k+1}) \leq f(x_k) + \gamma\left(\frac{L}{2}\gamma - 1\right)\|\nabla f(x_k)\|^2 \\ \iff & f(x_{k+1}) \leq f(x_k) + \frac{1}{L}\left(\frac{1}{2} - 1\right)\|\nabla f(x_k)\|^2 \\ \iff & f(x_{k+1}) \leq f(x_k) - \frac{1}{2L}\|\nabla f(x_k)\|^2 \\ \iff & f(x_{k+1}) - f(x^*) \leq f(x_k) - f(x^*) - \frac{1}{2L}\|\nabla f(x_k)\|^2 \end{aligned}$$

[2] **Editor's note.** The notes write this inequality with norms instead of *squared* norms: $\|x_{k+1} - x^*\| \leq \|x_k - x^*\| - \frac{1}{L^2}\|\nabla f(x_k)\|^2$. The squares come from the expansion of $\|x_{k+1} - x^*\|^2$ above. The conclusion is the same.

[3] **Editor's note.** The notes say "the GD sequence $(x_k)$ is decreasing and converges towards $x^*$". What is proven here is that the *distance* $\|x_k - x^*\|$ is non-increasing ($x_k$ itself is a vector, it cannot be "decreasing"). That alone does not prove convergence; the convergence of $f(x_k)$ towards $f(x^*)$ is exactly what the next theorem proves, with its speed.

Using the definition of convexity (Th 1):

$$\begin{aligned}
\langle \nabla f(x_k), x^* - x_k \rangle &\leq f(x^*) - f(x_k) \\
\langle \nabla f(x_k), x_k - x^* \rangle &\geq f(x_k) - f(x^*) \\
\|\nabla f(x_k)\| \, \|x_k - x^*\| &\geq f(x_k) - f(x^*) \qquad \text{(Cauchy–Schwarz)} \\
\|\nabla f(x_k)\|^2 &\geq \frac{\big(f(x_k) - f(x^*)\big)^2}{\|x_k - x^*\|^2} \\
-\frac{1}{2L}\|\nabla f(x_k)\|^2 &\leq -\frac{1}{2L}\frac{\big(f(x_k) - f(x^*)\big)^2}{\|x_k - x^*\|^2}
\end{aligned}$$

So:

$$f(x_{k+1}) - f(x^*) \leq f(x_k) - f(x^*) - \frac{1}{2L}\frac{\big(f(x_k) - f(x^*)\big)^2}{\|x_k - x^*\|^2}$$

Let $\delta_k = f(x_k) - f(x^*)$ and $\beta = \dfrac{1}{2L\|x_0 - x^*\|^2}$. Since $\|x_k - x^*\| \leq \|x_0 - x^*\|$ (previous theorem):[4]

$$\begin{aligned}
&\iff \delta_{k+1} \leq \delta_k - \beta\,\delta_k^2 \\
&\iff \delta_{k+1} - \delta_k \leq -\beta\,\delta_k^2 \iff \delta_k - \delta_{k+1} \geq \beta\,\delta_k^2
\end{aligned}$$

Dividing by $\delta_k\,\delta_{k+1}$:

$$\begin{aligned}
&\frac{1}{\delta_{k+1}} - \frac{1}{\delta_k} \geq \beta\frac{\delta_k}{\delta_{k+1}} \geq \beta \qquad \left(\delta_k \geq \delta_{k+1} \implies \frac{\delta_k}{\delta_{k+1}} \geq 1\right) \\
\iff &\sum_{k=0}^{K-1}\left(\frac{1}{\delta_{k+1}} - \frac{1}{\delta_k}\right) \geq K\beta \qquad \text{(with } K \text{ the number of necessary steps to reach an error } \varepsilon_K\text{)} \\
&\iff \frac{1}{\delta_K} - \frac{1}{\delta_0} \geq K\beta \qquad (\delta_0 > 0) \\
&\implies \frac{1}{\delta_K} \geq K\beta \iff \delta_K \leq \frac{1}{\beta K} \\
\iff &f(x_K) - f(x^*) \leq \frac{2L\|x_0 - x^*\|^2}{K}
\end{aligned}$$

$$\varepsilon_K \leq \frac{2L\|x_0 - x^*\|^2}{K}$$

*Error at step K* — *Constant C* — *Number of steps*

$$K \geq \frac{C}{\varepsilon} \text{ steps are enough to reach an error } \varepsilon$$ [5]

[4] **Editor's note.** The notes define $\beta = \frac{1}{2L\|x_k - x^*\|^2}$. That $\beta$ would change at every step, and the sum over $k$ below needs a constant. Replacing $\|x_k - x^*\|$ by the larger $\|x_0 - x^*\|$ (allowed thanks to the previous theorem) gives a constant $\beta$; it is also the value the notes use at the end.

*Interpretation:*

- If we want to reduce the error $\varepsilon_K$ by 10, we need to multiply $K$ by 10.
- The number of steps is exactly inversely proportional to the error.

*Conclusion:*

The convergence of GD is **sub-linear** in the case of convex L-smooth functions, with a complexity of:

$$O\left(\frac{1}{\varepsilon}\right)$$

*Remarks:*

We can aim for better results in terms of speed, given these assumptions:

* If the function is $\mu$-strongly convex: GD converges at $O\big(\log(1/\varepsilon)\big)$.
* If the function follows the Polyak–Łojasiewicz condition: GD converges at $O\big(\log(1/\varepsilon)\big)$.

[5] **Editor's note.** The notes box the result as $K \leq \frac{C}{\varepsilon_K}$, which is algebraically equivalent to $\varepsilon_K \leq \frac{C}{K}$. The useful reading for the complexity is the other way around: to guarantee an error $\varepsilon$, it is *enough* to take $K = \frac{C}{\varepsilon}$ steps, hence $O\left(\frac{1}{\varepsilon}\right)$.

CHAPTER

# III

# Stochastic Gradient Descent

*Problematic:*

SGD is an enhancement of GD, mostly used when the objective (loss) function can be written as a sum function:

$$x_{k+1} = x_k - \gamma \nabla f(x_k) \qquad \text{with:} \qquad \boxed{f(x_k) = \frac{1}{n} \sum_{i=1}^{n} f_i(x_k)} \qquad [1]$$

So the goal here came from a remark: to compute all the $(x_k)$, we need to compute all the $\nabla f_i(x_k)$ and sum them up!

$$\nabla f(x_k) = \frac{1}{n} \sum_{i=1}^{n} \nabla f_i(x_k)$$

Pretty slow though, isn't it? So here comes the idea of SGD.

*Def: SGD:*

We start from the assumption that $i$ (the index of $f_i$) is a random variable that follows a uniform distribution. In other words:

$$I = \{1, 2, \ldots, n\}, \qquad I \sim \mathcal{U} \iff P(I = i) = \frac{1}{n}$$

Then the idea is to compute the step $x_{k+1}$ by only using one $\nabla f_i(x_k)$, randomly chosen:

$$x_{k+1} = x_k - \gamma \nabla f_i(x_k)$$

*Well, does it work?*

*Theorem:*

SGD is an "**unbiased**" estimator of GD.

*Proof:*

An estimator $\hat{\theta}$ that estimates $\theta$ is said to be "unbiased" iff:

$$E(\hat{\theta}) - \theta = 0 \implies \boxed{E(\hat{\theta}) = \theta}$$

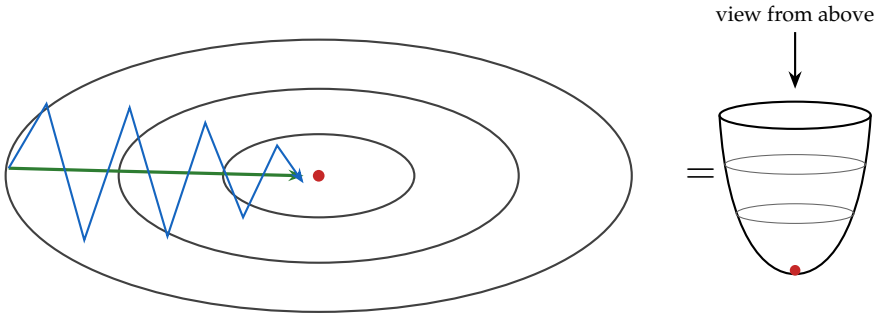
[1] **Editor's note.** The notes write the sum $\sum_{i=1}^{n-1}$, which has $n-1$ terms; with the factor $\frac{1}{n}$ the sum goes from $i = 1$ to $n$. The same fix was applied to the gradient, where the notes also forgot the $\nabla$: $\nabla f(x_k) = \frac{1}{n} \sum \nabla f_i(x_k)$.

So SGD relies on picking up $f_i$ randomly, with $P(I=i)=\frac{1}{n}$, then computing $\nabla f_i(x_k)$. So:

$$\begin{aligned} E\big(\nabla f_i(x_k)\,\big|\,x_k\big) &= \sum_{i=1}^{n} \nabla f_i(x_k) \times P(I=i) \\ &= \sum_{i=1}^{n} \nabla f_i(x_k) \times \frac{1}{n} = \frac{1}{n}\sum_{i=1}^{n} \nabla f_i(x_k) = \nabla f(x_k) \\ \Longrightarrow \underbrace{E\big(\nabla f_i(x_k)\,\big|\,x_k\big)}_{\text{leads to SGD}} &= \underbrace{\nabla f(x_k)}_{\text{leads to GD}} \end{aligned}$$

So SGD is an unbiased estimator of GD. ■

*Core idea and visualisation:*




ɯ : SGD ɯ : GD

As we see, it's a noisy pattern; the noise comes from the **variance**:

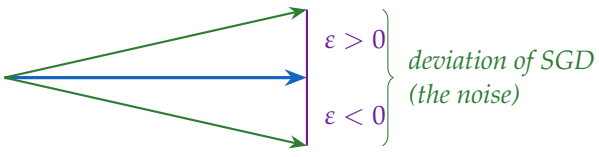
$$\begin{aligned} \mathrm{Var}_i\big(\nabla f_i(x_k)\big) &= E_i\Big(\big\|\nabla f_i(x_k) - E\big(\nabla f_i(x_k)\big)\big\|^2\Big) \\ &= E_i\Big(\big\|\underbrace{\nabla f_i(x_k) - \nabla f(x_k)}_{\varepsilon_i}\big\|^2\Big) \\ &= E_i\big(\|\varepsilon_i\|^2\big) \qquad \longleftarrow \text{ the deviation from the SGD point} \end{aligned}$$

The variance shows that in the SGD path, there are noises, so we can rewrite the SGD formula as follows:

$$\begin{aligned} x_{k+1} &= x_k - \gamma\big(\nabla f(x_k) + \varepsilon\big) \\ x_{k+1} &= x_k - \gamma\,\nabla f(x_k) - \gamma\,\varepsilon \qquad (\varepsilon \in \mathbb{R}^d) \end{aligned}$$

with $\varepsilon$ the noise, and $\boxed{E(\varepsilon)=0}$.[2]

So what happens:




ɯ $\nabla f_i(x_k)$ ɯ $\nabla f(x_k)$

[2] **Editor's note.** The notes write $\varepsilon \in \mathbb{R}$; the noise has the same dimension as the gradient, so $\varepsilon \in \mathbb{R}^d$.

⚠ *Important Remarks:*

1. In practice, we choose $f_i$ randomly, but we pass through all the $f_i$ with $i \in I$. The period of passing through all the $f_i$ is called an "**epoch**".
2. Complexity of SGD: $O\left(\frac{1}{\varepsilon^2}\right)$ for convex and L-smooth functions.[3]
   Slower (theoretically) than GD.

*Why, in practice, do we still prefer SGD to GD, even if it's slower and noisier than GD?*

The answer is: in practice,

$$\text{Cost} = O(c) \times t_{\text{iter}}$$

- $O(c)$: cost per iteration;
- $t_{\text{iter}}$: number of iterations.

Yes, it's true: $O\left(\frac{1}{\varepsilon}\right)$ is better than $O\left(\frac{1}{\varepsilon^2}\right)$. But don't forget that:

$$\underbrace{O(n)}_{\substack{\text{complexity to} \\ \text{compute } \nabla f(x_k)}} \text{ is slower than } \overbrace{O(1)}^{\substack{\text{complexity to} \\ \text{compute } \nabla f_i(x_k)}}$$

For a dataset with millions of rows, $n$ is huge: GD costs $O\left(\frac{n}{\varepsilon}\right)$, SGD costs $O\left(\frac{1}{\varepsilon^2}\right)$, and it doesn't depend on $n$.

*How about noises?*

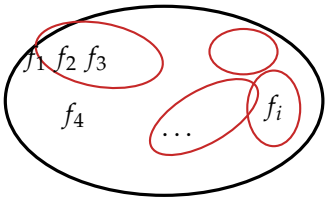
Noises aren't always bad! They can avoid local minima and saddle points problems, by taking other random paths than the straightforward one (for non-convex functions).

That's the main reason, but surely there are others, like avoiding overfitting since the algorithm is seeing new data at each step, etc.

## III.1 *Key Improvements*

### *Mini-Batch GD*

Well, computing $x_{k+1}$ by picking one $f_i(x_k)$ each time can be pretty slow. So, let's say this time:




*We divide the set of $f_i$ into equal batches, and then:*

[3] **Editor's note.** The notes write $O\left(\frac{1}{\sqrt{\varepsilon}}\right)$. For convex L-smooth functions (with a bounded noise variance and a decreasing step), SGD reaches $E[f(\bar{x}_K)] - f(x^*) = O\left(\frac{1}{\sqrt{K}}\right)$, i.e. $O\left(\frac{1}{\varepsilon^2}\right)$ iterations. Note that $O\left(\frac{1}{\sqrt{\varepsilon}}\right)$ would be *faster* than GD's $O\left(\frac{1}{\varepsilon}\right)$, which contradicts "slower than GD"; with $\frac{1}{\varepsilon^2}$ the sentence becomes true.

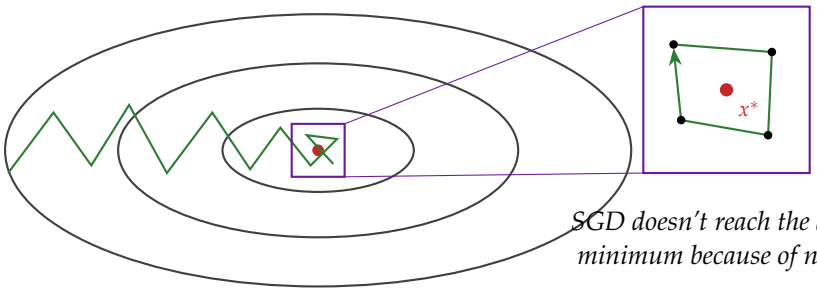
$$x_{k+1} = x_k - \gamma \nabla f_T(x_k) \qquad \text{with} \qquad \nabla f_T(x_k) = \frac{1}{\beta - \alpha} \sum_{i=\alpha}^{\beta-1} \nabla f_i(x_k)$$

with $\beta = \alpha +$ batch size.[4]

*That's the most used in practice.*

### *Average Stochastic Gradient Descent*

The ASGD comes from a discovery made by Boris Polyak:




*SGD doesn't reach the actual minimum because of noises.*

So:

$$x_k = x^* + \varepsilon_k \qquad \longleftarrow \text{ noises at iteration } k, \text{ with } E_k(\varepsilon) = 0$$

The idea: average it![5]

$$\bar{x}_K = \frac{1}{K} \sum_{k=0}^{K-1} x^* + \underbrace{\frac{1}{K} \sum_{k=0}^{K-1} \varepsilon_k}_{\approx E(\varepsilon_k)}$$

$$\bar{x}_K \approx x^* + \underbrace{E(\varepsilon_k)}_{=0}$$

$$\boxed{\bar{x}_K \approx x^*}$$

So then comes the algorithm:

- We first start with SGD:

$$x_k = x_{k-1} - \gamma \nabla f_i(x_{k-1})$$

- Then fix it with the Polyak average:

$$\bar{x}_k = \bar{x}_{k-1} + \frac{1}{k}(x_k - \bar{x}_{k-1})^6$$

[4] **Editor's note.** The notes sum from $i = \alpha$ to $\beta$, which gives batch size $+ 1$ terms while dividing by $\beta - \alpha =$ batch size. The upper index is $\beta - 1$.

[5] **Editor's note.** The notes write the average with the indices $\sum_{k=1}^{K-1}$ and $\frac{1}{K}$, and then an equality $\bar{x}_K = x^* + E(\varepsilon_k)$. The average of $K$ terms goes from $k = 0$ to $K - 1$; and the empirical mean of the noises is only *approximately* equal to its expectation (law of large numbers), so the result is $\bar{x}_K \approx x^*$, better and better as $K$ grows.

[6] **Editor's note.** The notes write $x_k = x_{k-1} - \gamma \nabla f_i(x_k)$; the gradient is computed at the current point $x_{k-1}$. The second line is the running mean: it is exactly $\bar{x}_k = \frac{1}{k} \sum_{j=1}^{k} x_j$, updated without storing all the $x_j$.

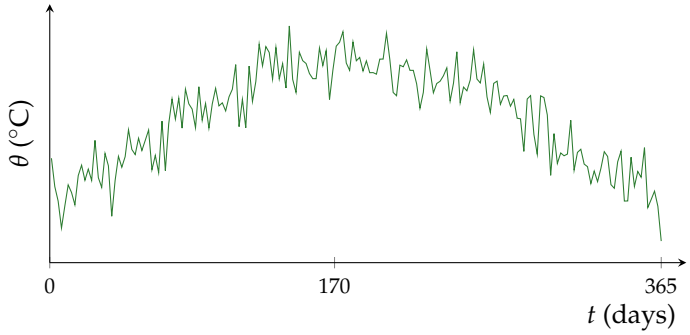
CHAPTER

# IV

# Momentum and Nesterov

## IV.1 Exponentially Weighted Average

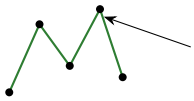
LET'S say we want to represent the temperature in Casablanca for the whole year:




When we interpolate (green curve) the data points, we get a noisy pattern. But even with this noisy pattern, we can globally know the global pattern of the curve: it's increasing, reaching a maximum, then decreasing.

So, to avoid noises and keep only the pattern, we should remark something:

*Where do noises come from?:* When we try to interpolate, we link data points, for example:



*This provokes noises: brutal deviations that come at once.*

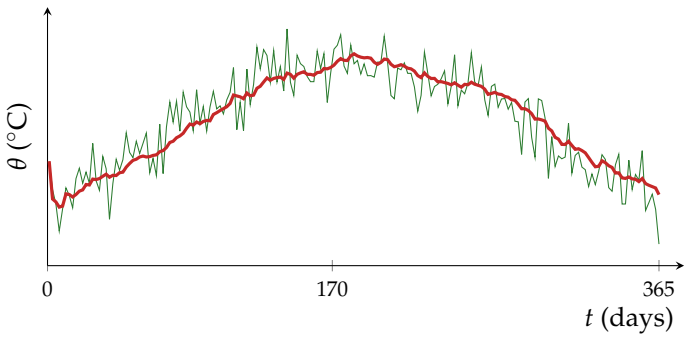
So, if we find a solution to avoid considering brutal data deviations, by giving much importance to the past evolution... let me introduce you:

*Def: Exponentially Weighted Average:*

EWA is defined as an iterative weighted average:

$$v_t = \beta\, v_{t-1} + (1-\beta)\, \theta_t$$

where $(v_t)$ is the representative sequence, and $\theta_t$ the function that we want to make smooth, while $\beta$ is a number $\in ]0,1[$ that manages the degree of smoothness. But the main thing is that $\beta$ **controls how many steps we want to memorize from the past**, so as to avoid brutal deviation problems.




The red curve represents the EWA effect for $\beta = 0.9$.

*Remark:* As a rule of thumb, very used in ML and DL:

$$N = \frac{1}{1-\beta} \qquad \text{with } N\text{: the number of remembered steps.}$$

For example, $\beta = 0.9$: $N = \frac{1}{1-0.9} = 10$ steps.

And we can assume that:

$$\boxed{\beta = 1 - \frac{1}{N}} \implies \text{if we want to remember } N \text{ steps, this is } \beta.$$

For example, we want to remember 100 steps: $\beta = 1 - \frac{1}{100} = 0.99$.

But let us see something very interesting:

$$\begin{aligned} v_{t-1} &= \beta v_{t-2} + (1-\beta)\theta_{t-1} \\ v_t &= \beta v_{t-1} + (1-\beta)\theta_t \\ v_{t+1} &= \beta v_t + (1-\beta)\theta_{t+1} \\ \implies v_{t+1} &= (1-\beta)\theta_{t+1} \underset{\times\beta}{\longrightarrow} (1-\beta)\beta\,\theta_t \underset{\times\beta}{\longrightarrow} (1-\beta)\beta^2\,\theta_{t-1} + \cdots \end{aligned}$$

(the three terms are summed). So we multiply by $\beta$ at each iteration.

*But when is it forgotten?*

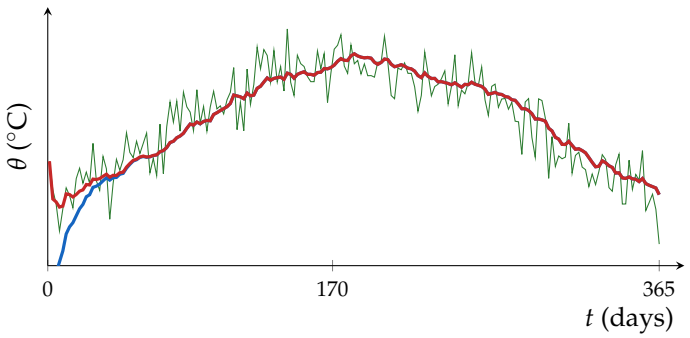
$$\beta = 1 - \frac{1}{N} \implies \beta^N = \left(1 - \frac{1}{N}\right)^N, \qquad \lim_{N\to+\infty} \beta^N = \lim_{N\to+\infty} \left(1 - \frac{1}{N}\right)^N = \frac{1}{e}$$

So if we choose $\beta$ depending on $N$, $\beta = 1 - \frac{1}{N}$, then the contribution after $N$ steps is $\frac{1}{e} \approx 37\%$.

*That's where the "exponential" comes from in EWA.*

*Remark: Bias Correction*

The red curve shown earlier isn't just an EWA, but an EWA *with bias correction*:




The blue curve shows what happens when we don't do bias correction, and the reason is that, theoretically and also in practice, we initialize $v_0 = 0$. But the problem is:

$$v_1 = 0 + (1 - \beta)\,\theta_1$$

In our example, where $\theta_t$ is the temperature and $\beta = 0.9$, let's say $\theta_1 = 11°\text{C}$. Then $v_1 = 0.1 \times 11 = 1.1°\text{C} \ll 11°\text{C}$, so we start far away from the starting point.

To correct that, we change the EWA formula:

*This is Bias Correction*

$$\hat{v}_t = \frac{v_t}{1 - \beta^t} = \frac{\beta v_{t-1} + (1 - \beta)\theta_t}{1 - \beta^t}$$

With $t = 1$: $\hat{v}_1 = \frac{1.1}{1-0.9} = 11°\text{C}$, exactly $\theta_1$. And since $\beta^t \to 0$, the correction disappears by itself after a few steps.

People don't mind fixing that in practice. Even libraries like PyTorch or TensorFlow don't add bias correction for momentum, since data scientists consider it as a "warm-up".[1] But for optimizers that use multiple moving averages (like Adam), bias correction is **mandatory**.[2]

## IV.2 *Momentum*

*Def: Momentum: "Persist in directions that keep reinforcing themselves."*

As you may have remarked, the squiggly line represented before looks the same as the SGD noisy pattern. So we apply EWA on the $\nabla f_i(x_k)$ (in practice $\nabla f_T(x_k)$ for mini-batch GD).

So the algorithm is as follows:

$$\begin{cases} v_t = \beta\, v_{t-1} + (1 - \beta)\, \nabla f_t(x_t) \\ x_{t+1} = x_t - \gamma\, v_t \end{cases}$$

[1] **Editor's note.** In fact PyTorch avoids the problem differently: the momentum buffer is initialised with the first gradient ($v_1 = g_1$) instead of 0.

[2] **Editor's note.** The notes cite "RMSProp/Adam". Adam applies bias correction (it's in the algorithm, see Chapter VI), but the classical RMSProp, and its PyTorch/TensorFlow implementations, do not.

*Remark:* Some AI engineers, and some frameworks like PyTorch, use this form of GD with momentum:

$$\begin{cases} v_t = \beta\, v_{t-1} + \nabla f_t(x_t) \\ x_{t+1} = x_t - \gamma\, v_t \end{cases}$$

And this is the reason why. Let's take our classical form, applied on GD ($g_t(x) = \nabla f_t(x)$):

$$\begin{aligned} v_t &= \beta v_{t-1} + (1-\beta) g_t(x) \\ \text{with:}\quad v_{t-1} &= \beta v_{t-2} + (1-\beta) g_{t-1}(x) \\ \text{so:}\quad v_t &= (1-\beta)\big(g_t(x) + \beta\, g_{t-1}(x) + \beta^2 g_{t-2}(x) + \cdots\big) \end{aligned}$$

So $g_t(x) = \nabla f_t(x)$ is scaled by:[3]

$$(1-\beta)\sum_{i=0}^{+\infty}\beta^i = (1-\beta)\times\frac{1}{1-\beta} = 1 \qquad \text{(geometric series, converges because } \beta < 1)$$

- So the gradient is always (approximately) scaled by 1.

Let's take the other formula, $v_t = \beta v_{t-1} + g_t(x)$. That leads to $\nabla f(x)$ being scaled by:

$$\sum_{i=0}^{+\infty}\beta^i = \frac{1}{1-\beta}$$

This is huge! Imagine if $\beta = 0.9$: $\frac{1}{1-0.9} = 10$, so the gradient is scaled by $10\times$, much faster! (Of course, we need to tune the learning rate.)

That's why engineers prefer this formula to the other one.

> *Important remark:* As I said, in PyTorch we don't use the form with $1-\beta$, but that's the default case. The truth is that, for PyTorch, the formula is as follows:
>
> $$\begin{cases} v_t = \beta\, v_{t-1} + (1-d)\, \nabla f_t(x_t) \\ x_{t+1} = x_t - \gamma\, v_t \end{cases}$$
>
> with $d$ the "**dampening**" parameter; by default, $d = 0$.

*You said before that noises could be helpful; why use an algorithm that smoothens noises?*

Well, good question! And the idea is, as I said, that the main consequence of noises is avoiding local minima and saddle point problems, by following another path rather than taking the straightforward one.

Well, SGD with momentum does follow a different path from the straightforward one, but rather than losing time switching directions at each iteration, we follow a more stable, smooth line that follows the global evolution of SGD.

[3] **Editor's note.** The notes sum from $i = 1$; the geometric sum $\sum \beta^i = \frac{1}{1-\beta}$ starts at $i = 0$ (the current gradient has weight $\beta^0 = 1$). Same fix in the next formula.

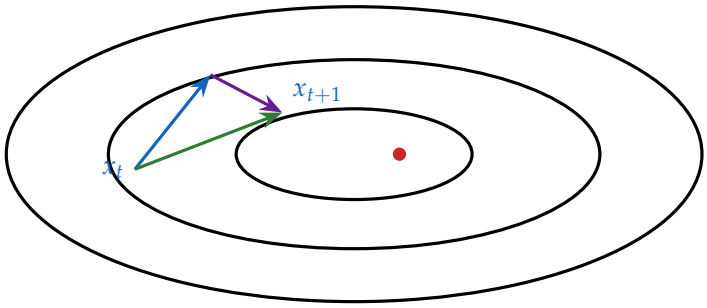
## IV.3 *Nesterov*

*Def: Nesterov: "See before you leap."*

Nesterov SGD is a smart idea found by Yurii Nesterov. The idea is: from the momentum formula,

$$v_t = \beta v_{t-1} + \nabla f_t(x_t), \qquad \text{then:} \qquad x_{t+1} = x_t - \gamma v_t,$$

let's see what happens:




$\wr\wr\, v_t \qquad \wr\wr\, \gamma\beta v_{t-1} \qquad \wr\wr\, \gamma\nabla f_t(x_t)$

So, what we can conclude from the figure is that $\beta v_{t-1}$ already gives us an idea of the region where we would "leap". So why not take it into consideration, and adjust the gradient at that point?

So here comes the Nesterov solution:

$$\begin{aligned} \bar{x}_t &= x_t - \gamma\beta\, v_{t-1} && \text{"See"} \\ v_t &= \beta v_{t-1} + \nabla f_t(\bar{x}_t) && \text{"Before"} \\ x_{t+1} &= x_t - \gamma\, v_t && \text{"You leap"} \end{aligned}$$

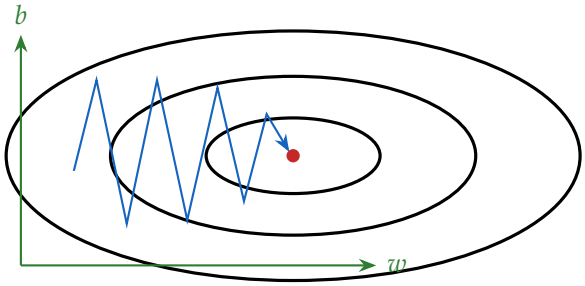
CHAPTER

# V

# RMSProp

As we have seen before with momentum, with the EMA we turned a noisy pattern into a smooth one. RMSProp relies on the same idea, but with only one important difference.[1]

The idea of RMSProp comes from this figure:




So the remark here is that, at each step, the gradient is bigger towards $b$, and less important towards $w$. We actually want it to go in the direction of $w$! So to solve this, we introduce the RMSProp algorithm:

*Def: RMSProp: "Normalise each direction by its historical volatility."*

RMSProp stands for **R**oot **M**ean **S**quare **Prop**agation, and this is the idea:

Square

Mean → $v_t = \beta v_{t-1} + (1-\beta)\,\nabla f_t^2(x_t)$ ⚠ Pointwise squaring! $\nabla f_t(x_t) \odot \nabla f_t(x_t)$

$$x_{t+1} = x_t - \gamma \frac{\nabla f_t(x_t)}{\sqrt{v_t} + \varepsilon}$$

⚠ "Pointwise root"

Root

*Well, what does this mean, and how would that help us reduce the effect of large gradients towards b?*

Let's see the details:

$$v_t = \beta v_{t-1} + (1-\beta)\,\nabla f_t^2(x_t)$$

[1] **Editor's note.** EMA (exponential moving average) and EWA (exponentially weighted average) are two names for the same thing; the notes use both.

$$\swarrow \qquad\qquad \searrow$$

$$v_{t,b} = \beta v_{t-1,b} + (1-\beta)\left(\frac{\partial f}{\partial b}\right)^2 \quad \Big| \quad v_{t,w} = \beta v_{t-1,w} + (1-\beta)\left(\frac{\partial f}{\partial w}\right)^2$$

If the step is bigger towards $b$:

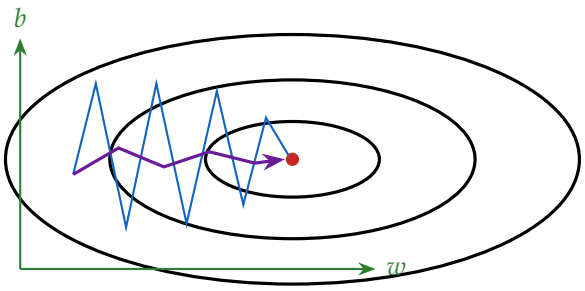
$$\left(\frac{\partial f}{\partial b}\right)^2 > \left(\frac{\partial f}{\partial w}\right)^2 \implies v_{t,b} > v_{t,w} \implies \sqrt{v_{t,b}} > \sqrt{v_{t,w}}$$

And to compute the step:

$$\begin{cases} x_{t+1,b} = x_{t,b} - \gamma \dfrac{\partial f/\partial b}{\sqrt{v_{t,b}} + \varepsilon} \\ x_{t+1,w} = x_{t,w} - \gamma \dfrac{\partial f/\partial w}{\sqrt{v_{t,w}} + \varepsilon} \end{cases}$$

*We divided by something bigger than this one.*

So if the gradient is big in a direction, we divide by a big quantity to reduce it, and that leads to this:




← *Propagation*

ɯ : SGD ɯ : RMSProp

*Another thing: what's "ε"?*

Epsilon plays a crucial role. It has two big roles:

1/ **Avoid dividing by zero!** (In practice we take $\varepsilon = 10^{-8}$; PyTorch: $\varepsilon = 10^{-8}$, TensorFlow: $\varepsilon = 10^{-7}$.)

2/ **Defining a minimum of division.** Imagine $\sqrt{v_t} \simeq 10^{-20}$.

- Without ε: $\dfrac{\nabla f_t(x_t)}{10^{-20}} = \nabla f_t(x_t) \times 10^{20}$ "Explosion".
- With ε: $\dfrac{\nabla f_t(x_t)}{\varepsilon + 10^{-20}} \simeq \dfrac{\nabla f_t(x_t)}{\varepsilon}$ (if $\varepsilon = 10^{-8}$: $10^{-8} + 10^{-20} \simeq 10^{-8}$).

Those are the main roles of ε.

*One last question: why do we divide by $\sqrt{v_t}$ and not $v_t$?*

And the answer comes from maths. With $v_0 = 0$:[2]

$$\begin{aligned} v_t &= \beta v_{t-1} + (1-\beta)\,\nabla f_t^2(x_t) \\ \implies v_t &= (1-\beta)\sum_{k=0}^{t-1} \beta^k\,\nabla f_{t-k}^2(x_{t-k}) \\ E(v_t) &= (1-\beta)\sum_{k=0}^{t-1} \beta^k\,E\big(\nabla f_{t-k}^2(x_{t-k})\big) \end{aligned}$$

Here comes the assumption of optimization in deep learning; we assume that:

$$E\big(\nabla f_t^2(x_t)\big) \simeq E\big(\nabla f_{t-1}^2(x_{t-1})\big) \simeq E\big(\nabla f^2(x)\big)$$

That leads to:

$$\begin{aligned} E(v_t) &\simeq (1-\beta) \times \frac{1-\beta^t}{1-\beta}\,E\big(\nabla f^2(x)\big) \\ \beta < 1 \implies \lim_{t\to+\infty} \beta^t &= 0 \\ \boxed{E(v_t) \simeq E\big(\nabla f^2(x)\big)} \end{aligned}$$

And for the $w$ direction:

$$E(v_{t,w}) \simeq E\left(\left(\frac{\partial f}{\partial w}\right)^2\right) \qquad \implies \qquad [v_{t,w}] \simeq \frac{[f]^2}{[w]^2}$$

(where $[\,\cdot\,]$ is the unit, the "dimension").

So, let's say we don't compute the root:

$$[x_{t+1,w}] = [x_{t,w}] - \gamma\,\frac{\dfrac{[f]}{[w]}}{\dfrac{[f]^2}{[w]^2} + \varepsilon}$$

*Dimension problem*

But with the root, $\left[\sqrt{v_{t,w}}\right] = \dfrac{[f]}{[w]}$:

$$[x_{t+1,w}] = [x_{t,w}] - \gamma\,\frac{\dfrac{[f]}{[w]}}{\dfrac{[f]}{[w]} + \varepsilon}$$

*Fixed!*

The ratio has no unit anymore: the step only depends on $\gamma$, whatever the scale of $f$ or of $w$.[3]

[2] **Editor's note.** The notes write the sum up to $+\infty$ and then use the finite sum $\frac{1-\beta^t}{1-\beta}$. Starting from $v_0 = 0$, $v_t$ has exactly $t$ terms, from $k = 0$ to $t-1$.

[3] **Editor's note.** Sentence added to make the conclusion of the dimensional analysis explicit.

CHAPTER

# VI

# *Adam and AdamW*

## VI.1 *Adam*

EVEN if Momentum and RMSProp seem perfect, since they solved the noisy SGD pattern problem in an elegant way, they both have problems.

*First, for Momentum:*

An objective function (or loss function) isn't always a calm, perfect convex function; it can be a steep valley, which could lead to having a gradient looking like this:

$$\nabla g(x) = \begin{pmatrix} 10000 \\ 1 \end{pmatrix}$$

Let's say we have a learning rate $\gamma = 0.001$:

$$\gamma \nabla g(x) = \begin{pmatrix} 10 \\ 0.001 \end{pmatrix} \begin{matrix} \leftarrow \text{ big step} \\ \leftarrow \text{ tiny step} \end{matrix}$$

So the gradient takes a big step in one direction, and a smaller one in another direction. Momentum's problem is that it can't notice it.

Solution? RMSProp, as explained before.

*The RMSProp problem:*

> *"RMSProp doesn't remember the direction that SGD is consistently pointing toward."*

*Demo:*

As we know, RMSProp also relies on EWA, so:[1]

$$v_T = (1-\beta) \sum_{k=0}^{T-1} \beta^k \, g_{T-k}^2$$

with $g_k$ the randomly chosen gradient:

$$g_k = \underbrace{g^*}_{\text{real gradient}} + \underbrace{\varepsilon}_{\text{noise}}$$

[1] **Editor's note.** The notes use $t$ both as the index of the sum and as the current step; the sum index was renamed $k$, and the gradients are written $g_k$ in the sum.

$$\iff v_T = (1-\beta)\sum_{k=0}^{T-1}\beta^k\left(g^{*2}+\varepsilon^2+2g^*\varepsilon\right)$$

$$\iff E[v_T] = (1-\beta^T)\Big(E\left[g^{*2}\right]+\mathrm{Var}[\varepsilon]\Big)$$

With bias correction:

$$\boxed{E(\hat{v}_T) = E\left(g^{*2}\right)+\mathrm{Var}(\varepsilon)}$$

This means that, for RMSProp, on average, $v_t$ follows the direction of the pointwise squared gradient. This means that if the gradient was pointing to:

$$(10;\ -10;\ 10;\ -10;\ 10;\ \dots) \qquad \text{(oscillation)}$$

RMSProp will remember:

$$(100;\ 100;\ 100;\ 100;\ \dots)$$

And the same if the gradient was consistently $(10; 10; 10; \dots)$: it doesn't know where the gradient was pointing to.

Therefore, Momentum:

$$v_T = (1-\beta)\sum_{k=0}^{T-1}\beta^k\, g_{T-k}, \qquad g_k = g^* + \varepsilon$$

$$v_T = (1-\beta)\sum_{k=0}^{T-1}\beta^k\,(g^*+\varepsilon)$$

$$E(v_T) = (1-\beta)\sum_{k=0}^{T-1}\beta^k\,E(g^*)$$

$$E(v_T)\simeq E(g^*) \implies \text{on average, Momentum follows the SGD's path.}$$

*What's the solution then? Why not combine Momentum and RMSProp?*

*Def: Adam: "Use Momentum for direction and RMSProp for scale."*

Adam stands for **Ada**ptive **M**oment estimation, and its algorithm is as follows:

$$\begin{cases} v_t = \beta_1 v_{t-1} + (1-\beta_1)\,\nabla f_t(x_t) \\ s_t = \beta_2 s_{t-1} + (1-\beta_2)\,\nabla f_t^2(x_t) \end{cases}$$

Moments
$E(v_t)\simeq E\left(\nabla f_t(x_t)\right)$
$E(s_t)\simeq E\left(\nabla f_t^2(x_t)\right)$

⚠ *Bias correction is mandatory for both $v_t$ and $s_t$!*

$$\hat{s}_t = \frac{s_t}{1-\beta_2^t} \qquad \text{and} \qquad \hat{v}_t = \frac{v_t}{1-\beta_1^t}$$

Both start from 0, and $v_t$ (with $\beta_1 = 0.9$) warms up faster than $s_t$ (with $\beta_2 = 0.999$, it takes about 1000 steps!). Without correction, $s_t$ stays close to 0 at the beginning, and dividing by $\sqrt{s_t}$ would make the first steps huge.[2]

[2] **Editor's note.** The notes say bias correction is needed "for the only reason that $v_t$ warms up faster than $s_t$". The difference of speed matters, but the root cause is that both averages start at 0 and are biased towards 0; the slow one ($s_t$) is the dangerous one, because it is in the denominator. The last sentence of the box was added to make this explicit.

$$x_{t+1} = x_t - \alpha \frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \varepsilon}$$

The authors of Adam advise to choose:

$$\beta_1 = 0.9 \qquad \text{and} \qquad \beta_2 = 0.999$$

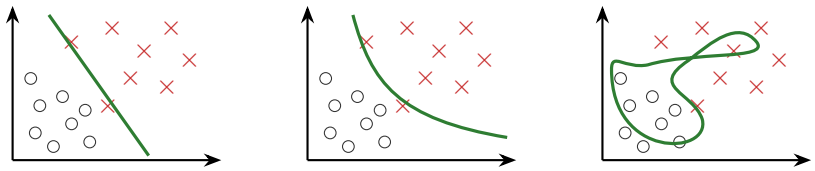
$\alpha$ should be tuned.[3]

So, as we can see, Adam solves both Momentum and RMSProp problems by combining them both. Even more, Adam is really fast, and that's why it's the most used optimizer; it's even the "default one".

## VI.2 *AdamW*

### *Intro to regularization:*

Regularization is one of the solutions that we may adopt when a model is overfitting. Here is the idea:



High bias (Under-fitting) | Perfect | High variance (Overfitting)

One of the causes that makes the model overfit the data is "giving too much weight to a neuron" (we will treat that deeply in Neural Networks). The solution is adding an $L_2$ penalty:

$$\tilde{L}(W) = L(W) + \lambda \|W\|_F^2$$

Remark: you may see $\|W\|_2^2$ for the Euclidean norm, but for neural networks weights are matrices, and for matrices we compute the "Frobenius norm":
$\|W\|_F = \sqrt{\sum_i \sum_j w_{ij}^2}$ [4]

So, like that, the model will try to avoid adding weights to neurons, since its goal is minimizing $\tilde{L}(W)$.

### *Is this related to Adam?*

[3] **Editor's note.** The paper (Kingma & Ba, 2014) proposes $\alpha = 0.001$ and $\varepsilon = 10^{-8}$ as defaults.

[4] **Editor's note.** The notes write $\|W\|_F = \left|\sum_i \sum_j w_{ij}^2\right|$, without the square root; the square root belongs to the norm, and it disappears in $\|W\|_F^2 = \sum_i \sum_j w_{ij}^2$, which is what the penalty uses.

Yes, and deeply. As I said, Adam leads to faster convergence. But applied to neural networks, if "adding weights" speeds up convergence, then Adam is happy and will do it, so we need to avoid that.[5]

The solution? Let me introduce:

*Def: AdamW: "Use Adam but with Weight Decay for generalization."*

AdamW has the same formula as Adam, but:

$$x_{t+1} = x_t - \alpha \frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \varepsilon} - \alpha\lambda x_t$$

*Weight decay (we will detail this more in the Regularization part)*

$$\implies x_{t+1} = (1 - \alpha\lambda)\, x_t - \alpha \frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \varepsilon}$$

When announced, AdamW was a huge discovery. It's commonly used for LLMs and large neural nets.

[5] **Editor's note.** Why can't we just add $\lambda\|W\|^2$ to the loss of Adam? Because Adam divides *every* gradient by $\sqrt{\hat{s}_t}$, including the gradient of the penalty ($2\lambda W$). The weights that had large gradients get a large $\hat{s}_t$, so they are regularized *less*, which is the opposite of what we want. AdamW (Loshchilov & Hutter) "decouples" the weight decay: it is applied directly to the weights, outside of the adaptive division. That's why the term $\alpha\lambda x_t$ below is not divided by $\sqrt{\hat{s}_t}$.

CHAPTER

# VII

# *Other Adam Versions and Improvements*

*You may ask: why are you still talking about optimizers? Aren't Adam and AdamW the best?*

Well, even if they are the most used optimizers, they still have some major problems.

## VII.1 *AdaMax*

*The Problem:*

AdaMax is a generalization of Adam, introduced in the same paper that talked about Adam. It solves a problem encountered by Adam. Let's look closely:

$$s_t = \beta_2 s_{t-1} + (1-\beta_2)\, \underbrace{g_t^2}_{\substack{\text{Pointwise squaring} \\ g_t^2 = g_t \odot g_t}}$$

Let's say:

$$g_t = \begin{pmatrix} 1000 \\ 1 \end{pmatrix} \implies g_t^2 = \begin{pmatrix} 10^6 \\ 1 \end{pmatrix} \quad \text{(very big gradient)}$$

For Adam, we use $\beta_2 = 0.999$; by the rule of thumb introduced before in Momentum:

$$N = \frac{1}{1-\beta_2} \quad \text{(number of remembered steps!)} \qquad N = \frac{1}{1-0.999} = 1000 \text{ remembered steps.}$$

*What does all of that mean?*

It means that large old gradients take time to vanish, and affect $s_t$ for a long time.

*Solution:*

The Adam researchers introduced a generalization of the second moment at the end of the paper. They said:

$$v_t = \beta_2^p\, v_{t-1} + (1-\beta_2^p)\, |g_t|^p$$

There comes the idea of AdaMax. Let's compute (writing $u_t = \lim_{p\to\infty} v_t^{1/p}$):[1]

$$\begin{aligned}
\lim_{p\to+\infty} v_t^{1/p} &= \lim_{p\to+\infty} \Big( (1-\beta_2^p)|g_t|^p + \beta_2^p v_{t-1} \Big)^{1/p} \\
&= \lim_{p\to+\infty} \Big( |g_t|^p + \big(\beta_2\, u_{t-1}\big)^p \Big)^{1/p} \\
&= \lim_{p\to+\infty} \big\| \big(|g_t|,\ \beta_2 u_{t-1}\big) \big\|_p \\
&= \big\| \big(|g_t|,\ \beta_2 u_{t-1}\big) \big\|_\infty \quad \longleftarrow \text{ the infinite norm}
\end{aligned}$$

$$\boxed{u_t = \max\big(|g_t|,\ \beta_2\, u_{t-1}\big)}$$

*Why does this solve the problem?*

Let's see closely. Let's say we had a big gradient, and let's compare the accumulated gradient to the new one ($\beta_2 = 0.999$):

$$\begin{cases} g_{t-1,w} = 100 \\ g_{t,w} = 20 \end{cases} \qquad u_t = \max\big(\beta_2 u_{t-1},\ g_{t,w}\big) = \beta_2 \times 100 = 99.9$$

$$\begin{cases} u_t = 99.9 \\ g_{t+1,w} = 90 \end{cases} \qquad u_{t+1} = \max(0.999 \times 99.9,\ 90) = 99.8$$

$$\vdots$$

$$\begin{cases} u_{N-1} \simeq 90.05 \\ g_{N,w} = 90 \end{cases} \qquad u_N = \max(0.999 \times 90.05,\ 90) = 90$$

$\uparrow$ old gradient completely vanished[2]

And for the algorithm:

① $\begin{cases} m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t \\ u_t = \max\big(\beta_2\, u_{t-1},\ |g_t|\big) \end{cases}$

② $\hat{m}_t = \dfrac{m_t}{1-\beta_1^t}$ (we don't need it for the second moment) [3]

③ $x_{t+1} = x_t - \alpha\, \dfrac{\hat{m}_t}{u_t}$

[1] **Editor's note.** The notes write the limit in a compact way ($\lim \|v_t\|_p = \|v_t\|_\infty$). Two details were made explicit: the factor $(1-\beta_2^p)^{1/p}$ tends to 1 when $p \to \infty$ (so it disappears), and the $p$-norm is the norm of the pair $(|g_t|,\ \beta_2 u_{t-1})$, whose infinite norm is their maximum.

[2] **Editor's note.** The notes end the example with $u_N = 72.9$ while the new gradient is 90. That can't happen: by construction $u_t = \max(\ldots, |g_t|) \geq |g_t|$, so $u_t$ never goes below the current gradient. Once the old maximum has decayed below 90 (after about $\ln(0.9)/\ln(0.999) \approx 105$ steps), $u_t$ simply becomes the current gradient, 90. The first steps were also recomputed: with $g_{t+1} = 90$, $u_{t+1} = \max(99.8, 90) = 99.8$. Compare with Adam: the $1000^2 = 10^6$ of the previous example never appears here, since AdaMax doesn't square the gradients.

[3] **Editor's note.** The notes write $u_t = \max(\beta_2|g_t|,\ u_{t-1})$: the factor $\beta_2$ is on the wrong term. It is the *old* value that decays, $\max(\beta_2 u_{t-1}, |g_t|)$, as derived just above.

As we can see, there's no need for "$\varepsilon$", since we are adding $|g_t| \implies u_t \neq 0$ (as long as the gradients are not all exactly 0). We don't need to compute the root either, for the same reason.

Even if this solution looks perfect, there's a problem: a maximum could be persistent for a long time. That could be bad, but sometimes it could be good, as we are going to see now.

## VII.2 *AMSGrad*

*Problem:*

The idea of AMSGrad is as follows. We divide by $\sqrt{s_t}$ for Adam; we add $\varepsilon$ to avoid dividing by 0. We usually choose $\varepsilon = 10^{-8}$, but if $s_t = 10^{-12} \implies \sqrt{s_t} = 10^{-6}$, and $10^{-6} + 10^{-8} \simeq 10^{-6}$. So:

$$x_{t+1} = x_t - \alpha \frac{\hat{v}_t}{10^{-6}} \implies x_{t+1} = x_t - \alpha \times \underbrace{10^6}_{\text{"Explosion"}} \hat{v}_t$$

The learning rate is used as a brake, to not overshoot the minimum. But now it's multiplied by $10^6$, which is big!

*Solution:*

Define a minimum of division, by defining:[4]

$$\hat{s}_t = \max\left(s_t,\ \hat{s}_{t-1}\right)$$

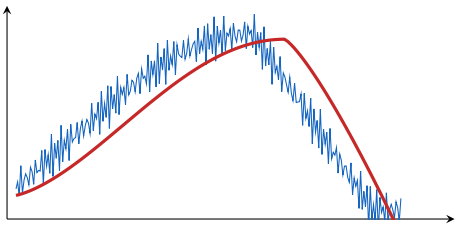
and then $x_{t+1} = x_t - \alpha \frac{\hat{v}_t}{\sqrt{\hat{s}_t} + \varepsilon}$: the denominator can never decrease, so the effective step $\alpha / \sqrt{\hat{s}_t}$ can never increase.

*Remark:* Remark that the researchers who made AMSGrad wanted to prove that Adam wasn't converging, even for simple convex function cases; that's why they introduced AMSGrad.

## VII.3 *QHAdam*

*Problem:*

Adam has a problem of adaptation to quick changes, since it relies on the EMA:



≀≀ : EMA

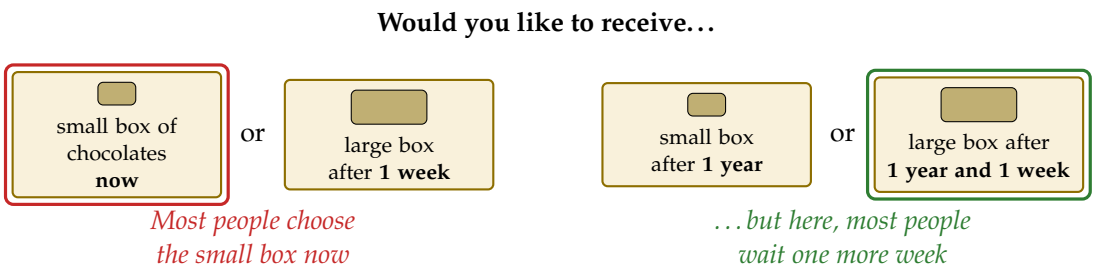
≀≀ : Noisy pattern

[4] **Editor's note.** The notes write $\hat{v}_t = \max(v_t; \hat{v}_{t-1})$. In this book $v_t$ is the first moment and $s_t$ the second moment (Chapter VI); the maximum is applied to the second moment, so it was written with $s$. In the AMSGrad paper the maximum is taken on $s_t$ (without bias correction).

So, by construction, "EMA-based algorithms" tend to slowly adapt to quick changes. There comes QHAdam.

*The solution:*

QHAdam stands for "**Q**uasi-**H**yperbolic **Adam**". And the meaning of "hyperbolic" comes from economics, especially the "*hyperbolic discounting*" concept,[5] which is to prioritize near and present events over future ones. This illustrates the idea perfectly:




So if we apply the idea to optimizers, it would be the exact solution: prioritize the present event. But if the present events are only noises, we should avoid that, by introducing a control variable "$\nu$". That's why it's "*Quasi*-Hyperbolic".

So the algorithm is as follows:

$$\textcircled{1}\quad \begin{cases} v_t = \beta_1 v_{t-1} + (1-\beta_1)\, g_t \\ s_t = \beta_2 s_{t-1} + (1-\beta_2)\, g_t^2 \end{cases} \qquad \textcircled{2}\quad \begin{cases} \hat{v}_t = \dfrac{v_t}{1-\beta_1^t} \\ \hat{s}_t = \dfrac{s_t}{1-\beta_2^t} \end{cases}$$

$$\textcircled{3}\quad \begin{cases} \tilde{v}_t = (1-\nu)\, g_t + \nu\, \hat{v}_t \\ \tilde{s}_t = (1-\nu)\, g_t^2 + \nu\, \hat{s}_t \end{cases} \qquad \textcircled{4}\quad x_{t+1} = x_t - \alpha \frac{\tilde{v}_t}{\sqrt{\tilde{s}_t} + \varepsilon}$$

[6]

## VII.4 *Yogi*

*Problem:*

The final one of our series of optimizers. It solves an important Adam problem. Let's see something:

$$\begin{aligned} & s_t = \beta_2 s_{t-1} + (1-\beta_2)\, g_t^2 \\ \iff\ & s_t - s_{t-1} = \beta_2 s_{t-1} - s_{t-1} + (1-\beta_2)\, g_t^2 \\ & \Delta s_t = -(1-\beta_2)\, s_{t-1} + (1-\beta_2)\, g_t^2 \\ & \Delta s_t = (1-\beta_2)\left(g_t^2 - s_{t-1}\right)^{7} \end{aligned}$$

[5] **Editor's note.** The notes write "Hyperparabolical discounting"; the concept is *hyperbolic* discounting. The notes illustrate it with an image from an article; it was redrawn here.

[6] **Editor's note.** The notes write $\tilde{s}_t = (1-\nu)g_t^2 + \nu v_t^2$; the second line mixes the *second* moment, so it is $\nu\, \hat{s}_t$. In the QHAdam paper (Ma & Yarats, 2019) the two lines have their own control variables, $\nu_1$ and $\nu_2$ (recommended: $\nu_1 = 0.7$, $\nu_2 = 1$, with $\beta_1 = 0.999$). With $\nu = 1$ we get back Adam; with $\nu = 0$, only the present gradient counts.

This is an actual problem, especially when $s_{t-1}$ is too large and $g_t^2$ is drastically too small. Then $g_t^2 - s_{t-1} \simeq -s_{t-1}$. For example, $s_{t-1,w} = 10^4$ and $g_{t,w} = 1$, so $g_{t,w}^2 - s_{t-1,w} = -9999$.

The change of $s_t$ then depends on $s_{t-1}$ itself, and not on the new gradients: $\Delta s_t \simeq -(1-\beta_2)\, s_{t-1}$, so $s_t$ *melts* exponentially. Don't forget that we divide by $\sqrt{s_t}$ in Adam, so the effective learning rate $\alpha / \sqrt{s_t}$ grows very quickly, and the steps can become dramatically big (and Adam can fail to converge).[8]

*Solution:*

Yogi (which stands for "to add" in the Sanskrit language) is the solution. Yogi simply says: "We don't care about $|g_t^2 - s_{t-1}|$, but only about its sign; the only important thing is $g_t^2$":

$$\begin{aligned} \Delta s_t &= (1-\beta_2)\ \mathrm{sign}\big(g_t^2 - s_{t-1}\big)\, g_t^2 \\ \iff s_t - s_{t-1} &= (1-\beta_2)\ \mathrm{sign}\big(g_t^2 - s_{t-1}\big)\, g_t^2 \end{aligned}$$

$$\iff s_t = s_{t-1} + (1-\beta_2)\ \mathrm{sign}\big(g_t^2 - s_{t-1}\big)\, g_t^2 \qquad [9]$$

With our example: $s_t = 10^4 + (1-\beta_2) \times (-1) \times 1$: $s_t$ moves by 0.001 instead of $\simeq 10$ for Adam.

And the other steps are like Adam (bias correction, compute the root, divide...).

[7] **Editor's note.** The notes use $v_t$ for Adam's second moment in this section; it was written $s_t$, as in Chapter VI.

[8] **Editor's note.** The notes conclude the opposite: that the step "becomes dramatically slow (since we divide the learning rate by a big number)". With $s_{t-1} \gg g_t^2$, Adam's $s_t$ *decreases* geometrically (it loses a fixed fraction $1-\beta_2$ of itself at each step), so the division becomes weaker and weaker, and the effective step *increases*, sometimes too fast. This is the problem described in the Yogi paper (Zaheer et al., 2018), and it is what Yogi's fix controls: its change is at most $(1-\beta_2)g_t^2$, so $s_t$ decreases slowly when the gradients are small.

[9] **Editor's note.** The notes write $\mathrm{sign}(g_t^2 - v_t)$; the sign compares the new squared gradient with the *previous* value, $s_{t-1}$ (the new one, $s_t$, is what we are computing).

CHAPTER

# VIII

# *Learning Rate Schedule*

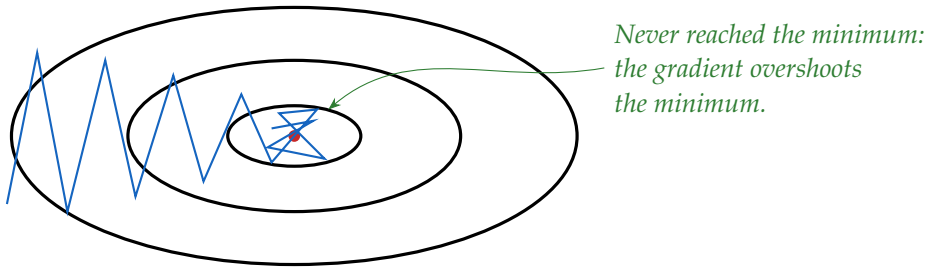
CONSIDER this part as the conclusion of this book.[1] I assume that you have been asking yourself, while learning about optimizers:

*How to choose $\alpha$, the learning rate?!*

To choose $\alpha$ isn't easy, and the answer is that there isn't one $\alpha$.

*Def 1: Constant learning rate:*

The constant learning rate is to choose $\alpha$ = constant for the whole training (let's say $\alpha = 0.1$). Let's look closely at what happens:




The idea is: $\alpha$ may be suitable at first, but near the minimum it overshoots, since the steps are too big.

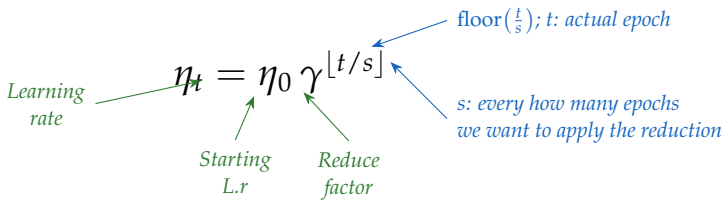
As an analogy, imagine you want to go to a store. At first, from your starting point, you drive fast (80 km/h); then, when you enter the street where the store is located, you brake to reduce the speed ($\sim$ 20 km/h); when you search where the store is, you go very slow ($\sim$ 7–5 km/h); then when you find it, you stop.

> *We need to create this braking system, not look for a store in a street at 80 km/h (constant learning rate).*

*Def 2: Constant learning rate decay (step decay):*

The idea is to reduce the learning rate constantly:

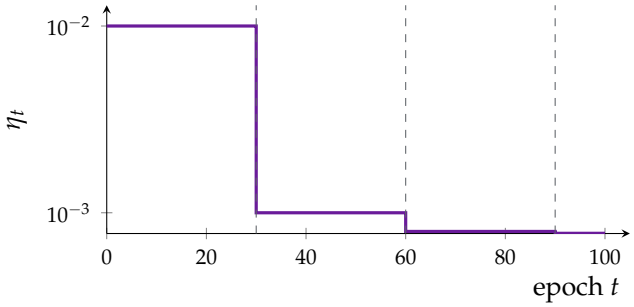
[1] **Editor's note.** The notes say "the conclusion of the first chapter": in the author's notebook, optimizers are the first chapter of a longer course.




For example, the sentence: "I want to reduce the learning rate 0.01 by a factor of $10^{-1}$ every 20 epochs" translates to:

$$\boxed{\eta_t = 0.01 \times 0.1^{\lfloor t/20 \rfloor}}$$

*Rule of thumb:* Reduce the learning rate at 30%, 60% and 90% of the training. For example, if the max epoch is 100: $s = 30$.




### *Def 3: Exponential decay:*

Exponential decay is like a smoother decay than the previous one:

$$\eta_t = \eta_0 e^{-kt} \qquad \longleftarrow \text{ } k\text{: a factor that needs to be defined}$$

To define $k$, there isn't a rule of thumb, but a more precise one:

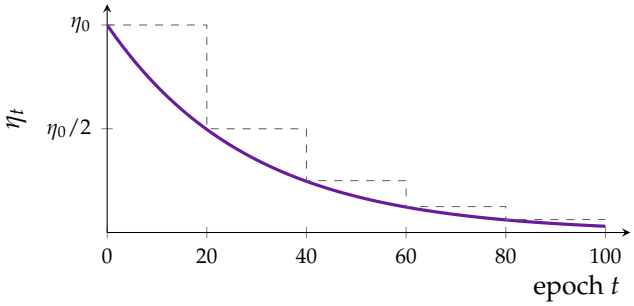
$$k = -\frac{\ln(\lambda)}{s}$$

$\lambda$: by how much we want to reduce the L.r

$s$: after how many epochs

Let's say we want to divide the l.r each time by 2 after 20 epochs:

$$k = -\frac{\ln(1/2)}{20} = \frac{\ln(2)}{20} \simeq 0.035 \qquad [2]$$

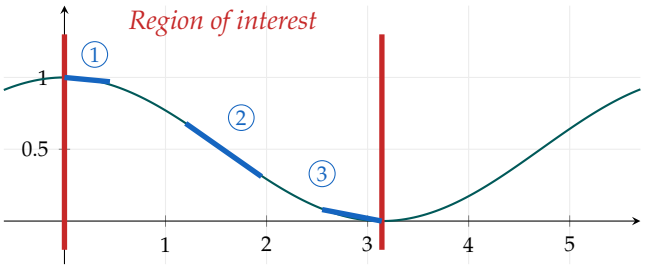



[2] **Editor's note.** Check: $e^{-0.035 \times 20} = e^{-0.69} \simeq 0.5$, the l.r is divided by 2 every 20 epochs, as wanted.

*Def 4: Cosine Annealing:*

The most used one. It follows the analogy perfectly: at first, nearly no braking, and at the end the braking. We follow the curve of:

$$\frac{1+\cos(x)}{2}$$




- Zone ①: nearly a horizontal line (no movement).
- Zone ②: linear straight line, not that harsh slope.
- Zone ③: a harsher, rude slope at the end.

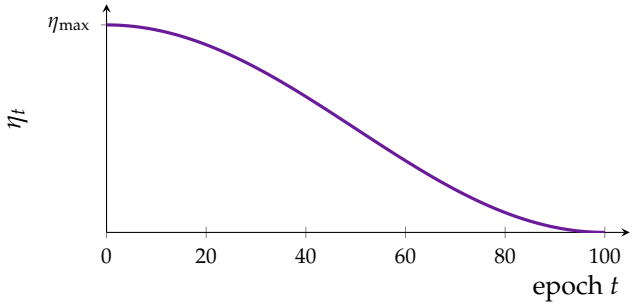
Project the tangent state on the brake state:

- Zone ①: nearly no braking.
- Zone ②: braking, but not too much.
- Zone ③: rude brake.

By changing the amplitude and the ending point:

$$\eta_t = \eta_{\min} + (\eta_{\max} - \eta_{\min}) \frac{1+\cos\left(\frac{t\pi}{T}\right)}{2}$$

- $T$: number of epochs;
- $t$: current epoch;
- $\eta_{\max}$: equivalent to $\eta_0$, "starting L.r";
- $\eta_{\min}$: desired ending "L.r". Sometimes chosen as 0, or $10^{-7} \sim 10^{-8}$: something too small.



*Def 5: Warm Up:*

> *The notes stop at the title "Def 5: Warm Up".* What follows was written during the typesetting, in the spirit of the notes, to complete the chapter.

Let's go back to our car. Every schedule we have seen starts at full speed: $\eta_0$ from the first epoch. But think about when you start your car in the morning: you don't leave the parking at 80 km/h. You don't know the road yet, the engine is cold, so you start slowly, and you accelerate only when everything is under control.

*Why would the first steps be dangerous for an optimizer?*

Remember what we said in the previous chapters:

- At the beginning, the weights are **random**: the model is completely wrong, the loss is big, and the gradients are big and chaotic. A big learning rate multiplied by a big gradient $\Longrightarrow$ a huge step, and the training can explode from the very first epochs.

- Adam divides by $\sqrt{\hat{s}_t}$, and at the beginning, $\hat{s}_t$ is estimated with only a few gradients (remember: with $\beta_2 = 0.999$, $s_t$ needs $\sim 1000$ steps to warm up). Bias correction fixes the *mean* of $s_t$, but not its *variance*: with 2 or 3 gradients, $\hat{s}_t$ is a very bad estimation, and $\frac{\alpha}{\sqrt{\hat{s}_t}}$ can be anything. That's also why, in Chapter IV, we said that data scientists consider the bias of the moving averages as "a warm up".

*Warm up: start with a small learning rate, and increase it progressively until reaching $\eta_{\max}$. Then, apply the decay.*

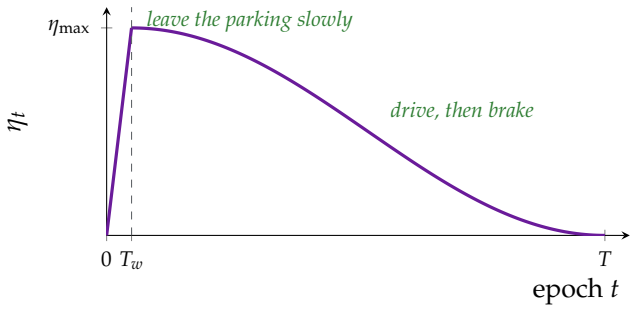
The most used one is the **linear warm up**: during the first $T_w$ steps (the "warm up steps"), we increase the learning rate linearly:

$$\eta_t = \eta_{\max} \times \frac{t}{T_w} \qquad \text{for } t \leq T_w$$

Then, after the warm up, we use one of the previous schedules, in practice the cosine annealing, on the remaining $T - T_w$ steps:

*Warm up + Cosine Annealing*

$$\eta_t = \begin{cases} \eta_{\max} \dfrac{t}{T_w} & t \leq T_w \\ \eta_{\min} + \frac{1}{2}\left(\eta_{\max} - \eta_{\min}\right)\left(1 + \cos\left(\pi \frac{t - T_w}{T - T_w}\right)\right) & T_w < t \leq T \end{cases}$$




**For example:** $T = 100$ epochs, $T_w = 5$ epochs, $\eta_{\max} = 10^{-3}$:

$$\eta_1 = 10^{-3} \times \frac{1}{5} = 2 \times 10^{-4}, \quad \eta_2 = 4 \times 10^{-4}, \quad \ldots, \quad \eta_5 = 10^{-3},$$

then the cosine brakes until $\eta_{100} = \eta_{\min}$.

*Rule of thumb:* The warm up takes a small part of the training, around 1% to 10% of the total steps. In practice, the warm up is counted in *steps* (mini-batches), not epochs: for big models, a few hundred to a few thousand steps.

*Remark:* Warm up was made famous by the Transformer paper (Vaswani et al., 2017, "Attention Is All You Need"), which used a linear warm up followed by a decay in $\frac{1}{\sqrt{t}}$. The RAdam paper (Liu et al., 2019) later explained it mathematically: it is exactly the variance of Adam's adaptive learning rate, during the first steps, that makes the warm up necessary.

*AdamW + Warm up + Cosine annealing:*
*this is the recipe used to train most of the modern large neural networks (LLMs).*

*So how do we choose $\alpha$, finally?*

We don't choose one $\alpha$, we choose a *trajectory* for $\alpha$: small at the start (warm up), big when the road is clear, and smaller and smaller when we get near the minimum (decay). Exactly like driving to the store.

# *Notes on this Edition*

This book is a faithful transcription of the author's handwritten notes. The wording, the examples and the order of the explanations are the author's. The typesetting only corrected grammar and spelling silently; everything that changes a formula, an index, a number, a name or a conclusion is listed below (and footnoted in the text as **Editor's note**).

### *Chapter I: Prerequisite*

- Th 1 needs $f$ differentiable. First-order Taylor remainder: $o(h)$, not $o(h^2)$. The converse of Th 1 was mentioned.
- The "global minimum formula" proves that a critical point is a global minimum, not that it is unique (that needs strict convexity).
- The limit $t \to 0$ was added in the definition of the Hessian (Th 2).
- Lipschitz constant: $L > 0$, not $L \in ]0, 1[$ (Def 2 and Th 3).
- Th 3 (descent lemma) is an implication of L-smoothness, not an equivalence; the " $\iff$ " after Cauchy–Schwarz are implications.
- "Def 4" renamed Th 4; the sign of (2) was fixed; the optimality condition in its proof is a vector equation, $z - y = -\frac{1}{L}(\nabla f(y) - \nabla f(x))$, not an equation on norms.

### *Chapter II: Gradient Descent*

- $\|f(x)\|^2 \to \|\nabla f(x)\|^2$ in the descent inequality.
- The inequality after co-coercivity is on *squared* norms.
- What decreases is the distance $\|x_k - x^*\|$, not "the sequence $(x_k)$".
- $\beta$ is defined with $\|x_0 - x^*\|$ (a constant), not $\|x_k - x^*\|$.
- The final bound is read as: $K = C/\varepsilon$ steps are *enough* to reach an error $\varepsilon$ (the notes box $K \leq C/\varepsilon_K$).

### *Chapter III: Stochastic Gradient Descent*

- Sums from $i = 1$ to $n$ (not $n - 1$); missing $\nabla$ in the full gradient; noise $\varepsilon \in \mathbb{R}^d$.
- Complexity of SGD for convex L-smooth functions: $O(1/\varepsilon^2)$, not $O(1/\sqrt{\varepsilon})$ (which would be faster than GD).
- Mini-batch sum up to $\beta - 1$; ASGD average from $k = 0$ to $K - 1$, with $\bar{x}_K \approx x^*$ (law of large numbers); gradient at $x_{k-1}$ in the SGD step.

*Chapter IV: Momentum and Nesterov*

- Geometric sums start at $i = 0$.
- Precisions: PyTorch initialises the momentum buffer with the first gradient; classical RMSProp does not use bias correction (Adam does).

*Chapter V: RMSProp*

- With $v_0 = 0$, $v_t$ is a finite sum from $k = 0$ to $t - 1$.

*Chapter VI: Adam and AdamW*

- Sum index renamed $k$ (it clashed with $t$).
- The reason for bias correction was made explicit (both averages start at 0; the slow one is in the denominator).
- Frobenius norm: $\|W\|_F = \sqrt{\sum\sum w_{ij}^2}$.
- Added: why weight decay must be decoupled from Adam's adaptive division.

*Chapter VII: Other Adam Versions and Improvements*

- AdaMax: $u_t = \max(\beta_2 u_{t-1}, |g_t|)$ (the notes put $\beta_2$ on $|g_t|$); details of the limit $p \to \infty$; the example ends at $u_N = 90$ (the notes' 72.9 is below the current gradient, which is impossible).
- AMSGrad: the maximum is on the second moment $s_t$.
- QHAdam: $\tilde{s}_t = (1 - \nu) g_t^2 + \nu \hat{s}_t$; "hyperbolic" discounting; the image of the notes was redrawn.
- Yogi: $\Delta s_t$ for the example is $-9999(1 - \beta_2)$; Adam's problem is that $s_t$ *decreases* too fast (the effective step *grows*), not that the step becomes slow; the sign compares $g_t^2$ with $s_{t-1}$.

*Chapter VIII: Learning Rate Schedule*

- **Completed during the typesetting** (the notes stop at the title): Def 5, Warm Up, with its motivation, the linear warm up formula, warm up + cosine annealing, an example, rules of thumb, and the plots of every schedule.

*From one step down the gradient*
*to the optimizers that train LLMs.*

Why is the learning rate of Gradient Descent $1/L$, and how fast does it converge? Why do we prefer Stochastic Gradient Descent even if it is slower and noisier? Where does the "exponential" of the exponentially weighted average come from? Why does RMSProp divide by a square *root*? And why did the world move from Adam to AdamW?

*The Enchiridion for AI Optimizers Theory* answers these questions one proof at a time: convexity and L-smoothness, Gradient Descent, SGD, Momentum and Nesterov, RMSProp, Adam and AdamW, AdaMax, AMSGrad, QHAdam and Yogi, and the learning rate schedules, from step decay to warm up and cosine annealing.

Written by hand by Reda Louriki, and typeset in LaTeX by Claude Opus 5.5.

REDA LOURIKI

THE ENCHIRIDION SERIES · BOOK II

